

JavaScript



¿Qué nos falta?

Hasta ahora construimos páginas Web usando:

- ▶ HTML para definir la estructura
- ▶ CSS para definir la presentación

Con esto podemos mostrar información... pero hay un problema importante:

Todo es estático

Si el usuario interactúa con la página:

- ▶ hace clic
- ▶ escribe en un campo
- ▶ envía un formulario

No ocurre nada por sí solo.

¿Qué nos falta? (Cont.)

Una interfaz sin comportamiento no es una aplicación... es solo un documento.

Para resolver esto aparece JavaScript.

No como un lenguaje más... sino como el mecanismo que permite que la interfaz reaccione.

Con JavaScript podemos:

- ▶ detectar acciones del usuario
- ▶ ejecutar lógica
- ▶ modificar lo que se muestra en pantalla

¿Dónde se ejecuta JavaScript?

A diferencia de otros lenguajes que vimos indirectamente...
JavaScript se ejecuta principalmente en: **el navegador**
(lado del cliente)

Esto significa que:

- ▶ el código corre en la máquina del usuario
- ▶ responde a sus acciones en tiempo real
- ▶ no necesita consultar al servidor para todo



Ejemplo simple

```
<button id="ibtn">Presionar</button>
```

```
<script>  
document.getElementById("ibtn").addEventListener("click", () => {  
    alert("¡Hola Mundo!");  
});  
</script>
```

Ejemplo simple: ¿Qué está pasando?

Cuando el usuario hace clic:

- ▶ el navegador detecta el evento
- ▶ JavaScript “escucha” ese evento
- ▶ se ejecuta una función
- ▶ ocurre una acción (mostrar mensaje “¡Hola Mundo!”)

El sistema reacciona a una acción del usuario.

¿Qué es JavaScript?

JavaScript es un lenguaje de programación que **permite agregar comportamiento a páginas Web**.

Es uno de los tres pilares del desarrollo Web junto con:

- ▶ HTML
- ▶ CSS

Se caracteriza por ser:

- ▶ **interpretado**
- ▶ **dinámico**
- ▶ **orientado a eventos**

Pero lo más importante no es cómo es... sino para qué lo usamos: **interacción**

JavaScript en el sistema

JavaScript no es solo un lenguaje de interfaz.

Es un lenguaje que puede ejecutarse en **distintos puntos del sistema.**

En el cliente (navegador)

Cuando se ejecuta en el navegador, JavaScript:

- ▶ interpreta y modifica el DOM
- ▶ responde a eventos del usuario
- ▶ valida datos antes de enviarlos
- ▶ gestiona el estado de la interfaz

Esto significa que parte de la lógica de la aplicación ya no depende del servidor.

JavaScript en el sistema (Cont.)

En el servidor

Con entornos como Node.js, JavaScript también **puede ejecutarse en el backend.**

En ese contexto, se encarga de:

- ▶ procesar solicitudes HTTP
- ▶ acceder a bases de datos
- ▶ aplicar reglas de negocio
- ▶ generar respuestas

Es decir, cumple el mismo rol que otros lenguajes tradicionales de servidor.

Formas de incorporarlo

- ▶ En línea (dentro de un atributo HTML):

```
<button onclick="alert('Hola')">Presionar</button>
```

- ▶ Embebido (dentro de la etiqueta <script>):

```
<script>  
function saludar() {  
    alert('Hola');  
}  
</script>
```

Formas de incorporarlo (Cont.)

► Archivo externo:

```
<script src="script.js"></script>
```

Alguno de sus atributos principales son:

- **async**: Carga el script de forma asíncrona con respecto al HTML. No garantiza el orden de ejecución. Su uso típico es cuando el script que no depende del DOM ni de otros scripts.
- **defer**: También carga el script de forma asíncrona, pero espera a que todo el HTML esté cargado antes de ejecutarse. Mantiene el orden si hay varios scripts con defer. Recomendado para scripts que interactúan con el DOM.

Tipos de datos

JavaScript es **un lenguaje de programación de tipo dinámico**, esto quiere decir que:

- ▶ **No es necesario declarar el tipo de dato de una variable:** JavaScript determina automáticamente el tipo de dato en el momento de la ejecución basándose en el valor que se le asigna a la variable.
- ▶ **Las variables pueden cambiar de tipo:** Una variable puede comenzar almacenando un valor de un tipo (por ejemplo, un número) y luego ser reasignada para almacenar un valor de otro tipo (por ejemplo, una cadena de texto).

Tipos de datos (Cont.)

- ▶ **Tipos primitivos:** Son **inmutables** y se almacenan directamente en la ubicación que la variable accede. Los valores primitivos no se pueden modificar después de su inicialización; cualquier operación que parezca modificar un valor primitivo en realidad crea un nuevo valor. Se controlan con el operador **typeof**. A continuación, los enumeramos:
 - ▶ *String*: Representa texto entre comillas. Por ejemplo, "¡Hola mundo!".
 - ▶ *Number*: Para números enteros o flotantes. JavaScript utiliza el formato de doble precisión de 64 bits IEEE 754. Por ejemplo, 42, 3.14.

Tipos de datos (Cont.)

- ▶ **Tipos primitivos:** Continuando con lo anterior:
 - ▶ *BigInt*: Para números enteros muy grandes que superan el límite de *Number*. Fue introducido en ECMAScript 2020. Por ejemplo, 9007199254740991n.
 - ▶ *Boolean*: Representa una entidad lógica que puede tener dos valores: *true* y *false*.
 - ▶ *Undefined*: Una variable sin un valor asignado tiene el valor *undefined*.
 - ▶ *Symbol*: Un valor único e inmutable, útil para claves de objeto que necesitan ser irrepetibles. Cada vez que se invoca *Symbol()*, se crea un nuevo y único símbolo, no hay dos símbolos que puedan ser iguales, incluso si tienen la misma descripción.

Tipos de datos (Cont.)

- ▶ **Tipos compuestos:** Los tipos compuestos (o de referencia) incluyen **objetos** y permiten almacenar **colecciones de datos** o **funciones** más complejas. Por ejemplo, un objeto literal: `{ nombre: "Ámbar", edad: 10 }`, un array: `[1, 2, 3]`, una función: `function miFuncion() {}`
- ▶ **Tipos especiales:** Estos tipos especiales tienen **comportamientos únicos** o sirven para propósitos específicos dentro del lenguaje.
 - ▶ *Null*: Denota un valor nulo o “vacío”. Muchas veces se lo considera como parte de los primitivos.
 - ▶ *Function*: Un bloque de código diseñado para realizar una tarea específica. Es un objeto subtipo que es ejecutable.

Declaración de variables

Una variable es un **contenedor para almacenar datos**. En JavaScript, se puede declarar variables usando **var**, **let** o **const**. La elección entre estos depende del ámbito y la reasignabilidad que se necesite.

- ▶ **var**: Declara una variable, opcionalmente inicializándola con un valor. Hoy no se recomienda su uso.
- ▶ **let**: Declara una variable local de ámbito de bloque, opcionalmente inicializándola con un valor.
- ▶ **const**: Declara una constante de ámbito de bloque, que debe ser inicializada con un valor en el momento de su declaración.

En la práctica actual, se prefiere **let** y **const** por su comportamiento más predecible.

Ámbitos de las variables

El ámbito (o scope) se refiere al **contexto en el que una variable es accesible**. JavaScript define principalmente dos tipos de ámbitos:

- ▶ **Ámbito global:** Una variable declarada fuera de cualquier función o bloque es una variable global, lo que significa que es accesible desde cualquier parte de tu código.
- ▶ **Ámbito local:** Las variables declaradas dentro de una función o bloque están limitadas a ese contexto y no son accesibles fuera de él.

Ámbitos de las variables (Cont.)

Hay dos tipos de **ámbitos locales** en JavaScript:

- ▶ **Ámbito de función:** Las variables declaradas con **var** dentro de una función están disponibles en toda la función, incluso dentro de bloques como **if**, **for**, etc. Sin embargo, no son accesibles fuera de la función. Esto puede provocar comportamientos inesperados si no se tiene cuidado.
- ▶ **Ámbito de bloque:** Las variables declaradas con **let** o **const** tienen un ámbito limitado al bloque **{}** donde fueron definidas (por ejemplo, dentro de un **if**, **for**, o función). No pueden ser accedidas fuera de ese bloque, lo cual evita errores y hace el código más seguro.

Declaración de funciones

Una declaración de función en JavaScript **define una función utilizando la palabra clave function**, seguida de un nombre, paréntesis que pueden contener parámetros, y un bloque de código que representa el cuerpo de la función.

```
function nombreDeLaFuncion(parametro1, parametro2) {  
    // Cuerpo de la función  
}
```

Declaración de funciones (Cont.)

Las funciones declaradas tienen un **ámbito léxico**, es decir, se definen dentro de un contexto que determina dónde pueden ser accedidas:

- ▶ Si una función se declara en el ámbito global, está disponible desde cualquier parte del código.
- ▶ Si se declara dentro de otra función, su alcance queda limitado al interior de esa función padre.

Declaración de funciones (Cont.)

Expresiones de Función

Además de las declaraciones tradicionales, JavaScript permite definir funciones mediante **expresiones de función**. Estas funciones pueden ser **anónimas** y se suelen asignar a variables o constantes.

Una diferencia clave es que **las expresiones de función no son elevadas (hoisted)** como las declaraciones, por lo tanto, **no se pueden invocar antes de su definición**.

```
const saludar = function(nombre) {  
    console.log("¡Hola " + nombre + "!");  
};  
saludar("Ana");
```

Declaración de funciones (Cont.)

Funciones Flecha (Arrow Functions)

Las funciones flecha ofrecen una **sintaxis más corta** para escribir expresiones de función. Son ideales como callbacks y funciones que se pasan como argumentos.

A diferencia de las funciones tradicionales, el valor de **this** en una función flecha se hereda del contexto en el que fue definida.

```
const sumar = (a, b) => a + b;  
sumar(7, 12);
```

Funciones predefinidas

También conocidas como **funciones globales se utilizan para realizar diversas tareas**, como trabajar con números, textos, fechas, temporizadores y mucho más. A continuación, algunas de ellas:

- ▶ **Funciones para trabajar con números:**

- ▶ *parseInt(string, base)*: Convierte una cadena a un entero en la base especificada.
- ▶ *parseFloat(string)*: Convierte una cadena a un número de punto flotante.
- ▶ *isNaN(value)*: Comprueba si el valor es NaN (Not-a-Number).
- ▶ *isFinite(value)*: Comprueba si el valor es un número finito.

Funciones predefinidas (Cont.)

► Funciones para trabajar con textos:

- *slice(start[, end])*: Extrae una sección de una cadena y devuelve una nueva sin modificar la original. Puede aceptar índices negativos (contando desde el final de la cadena).
- *substring(start[, end])*: Similar a *slice()*, pero no puede aceptar índices negativos y esta es la principal diferencia con la función anterior.

Funciones predefinidas (Cont.)

- ▶ **Funciones para trabajar con temporizadores:**
 - ▶ *setTimeout(function, delay, ...args)*: Ejecuta una función después de un retraso especificado (en milisegundos).
 - ▶ *setInterval(function, interval, ...args)*: Ejecuta repetidamente una función con un intervalo de tiempo fijo entre cada llamada.
 - ▶ *clearTimeout(timeoutId)*: Cancela un temporizador establecido con *setTimeout*.
 - ▶ *clearInterval(intervalId)*: Cancela un temporizador establecido con *setInterval*.

Funciones predefinidas (Cont.)

► Otras funciones útiles:

- *alert(message)*: Muestra un cuadro de alerta con un mensaje y espera que el usuario presione "Aceptar".
- *prompt(text[, defaultText])*: Muestra un cuadro de diálogo que solicita al usuario una entrada, con un texto de sugerencia opcional.
- *confirm(message)*: Muestra un cuadro de diálogo con un mensaje y dos botones: "Aceptar" y "Cancelar".

Funciones predefinidas (Cont.)

► Funciones de consola para depuración:

- `console.log(message, ...)`: Imprime un mensaje en la consola del navegador.
- `console.error(message, ...)`: Imprime un mensaje de error en la consola.
- `console.warn(message, ...)`: Imprime un mensaje de advertencia en la consola.
- `console.info(message, ...)`: Imprime un mensaje informativo en la consola.

Funciones predefinidas (Cont.)

► Funciones para trabajar con arrays:

- *slice(start[, end])*: Retorna una copia de una parte del array dentro de un nuevo. El array original no se modifica.
- *splice(start[, amount[, element1[, ...[, elementN]]]])*: Cambia el contenido de un array eliminando, reemplazando o añadiendo nuevos elementos. Este método modifica el array original.
- *map(function)*: Crea un nuevo array con los resultados de la llamada a la función proporcionada aplicada a cada elemento del array original.
- *filter(function)*: Crea un nuevo array con todos los elementos que cumplan la condición implementada por la función proporcionada.

Funciones predefinidas (Cont.)

- ▶ **Funciones para trabajar con arrays:** Continuando con lo anterior:
 - ▶ *reduce(function[, initialValue]):* Aplica una función a un acumulador y a cada valor del array (de izquierda a derecha) para reducirlo a un único valor.
 - ▶ *find(function):* Devuelve el valor del primer elemento del array que cumple la función de prueba proporcionada.
 - ▶ *indexOf(element):* Devuelve el primer índice en el que se puede encontrar un elemento dado en el array, o -1 si no está presente.
 - ▶ *includes(element):* Determina si un array incluye un determinado elemento, retornando true o false según corresponda.

Funciones predefinidas (Cont.)

- ▶ **Funciones para trabajar con arrays:** Continuando con lo anterior:
 - ▶ *sort(function)*: Ordena los elementos de un array in situ y devuelve el array.
 - ▶ *push(element1, ..., elementN)*: Añade uno o más elementos al final de un array y devuelve la nueva longitud del array.
 - ▶ *pop()*: Elimina el último elemento de un array y lo devuelve.
 - ▶ *unshift(element1, ..., elementN)*: Añade uno o más elementos al inicio del array y devuelve la nueva longitud del array.

Funciones predefinidas (Cont.)

- ▶ **Funciones para trabajar con arrays:** Continuando con lo anterior:
 - ▶ *shift()*: Elimina el primer elemento del array y lo devuelve.
 - ▶ *forEach(function)*: Ejecuta la función proporcionada una vez por cada elemento del array.

Estructuras de control

- ▶ **if:** Se utiliza para ejecutar un bloque de código si, y solo si, una condición específica es verdadera. También se puede expandir para incluir un bloque else que se ejecutará si la condición es falsa, y opcionalmente, else if para más condiciones:

```
if (condicion) {  
    // bloque de código que se ejecuta si "condicion" es verdadera  
} else if (otraCondicion) {  
    // bloque de código que se ejecuta si "otraCondicion" es verdadera  
} else {  
    // bloque de código que se ejecuta si todas las condiciones  
    anteriores son falsas  
}
```


Estructuras de control (Cont.)

- **switch:** Permite ejecutar diferentes bloques de código en función del valor de una expresión. Útil cuando se tienen múltiples casos contra una misma expresión.

```
switch (expresion) {  
    case valor1:  
        // Bloque de código a ejecutar si expresion === valor1  
        break;  
    case valor2:  
        // Bloque de código a ejecutar si expresion === valor2  
        break;  
    default:  
        // Bloque de código a ejecutar si expresion no coincide con ningún caso  
}
```

Estructuras de control (Cont.)

- ▶ **for:** Crea un bucle que consiste en tres expresiones opcionales, encerradas en paréntesis y separadas por puntos y coma, seguidas por un bloque de código a ejecutar en el bucle.

```
for (inicializacion; condicion; incremento) {  
    // Bloque de código a ejecutar  
}
```

- ▶ **while:** Crea un bucle que ejecuta un bloque de código mientras una condición especificada es verdadera.

```
while (condicion) {  
    // Bloque de código a ejecutar  
}
```

Estructuras de control (Cont.)

- ▶ **do...while:** A diferencia del bucle while, que evalúa la condición antes de ejecutar el bloque de código, el bucle do...while asegura que el bloque de código se ejecute al menos una vez, ya que la condición se evalúa después de la ejecución del bloque de código. Esto es útil en situaciones donde se quiere que el bloque de código se ejecute al menos una vez independientemente de la condición.

```
do {  
    // Bloque de código a ejecutar  
} while (condicion);
```

Estructuras de control (Cont.)

- ▶ **for...in:** Este bucle permite recorrer todas las propiedades enumerables de un objeto. En cada iteración, la variable utilizada representa el nombre de la propiedad (clave), no un índice numérico. Es especialmente útil cuando se trabaja con objetos cuyos nombres de propiedades no se conocen previamente.

```
for (variable in objeto) {  
    // Código a ejecutar  
}
```

Operadores

▶ Operadores Aritméticos:

- ▶ *Suma (+)*: Suma dos números o concatena cadenas.
- ▶ *Resta (-)*: Resta un número de otro.
- ▶ *Multiplicación (*)*: Multiplica dos números.
- ▶ *División (/)*: Divide un número entre otro.
- ▶ *Módulo (%)*: Devuelve el residuo de una división.
- ▶ *Incremento (++)*: Aumenta el valor de una variable en uno.
 - ▶ *Incremento Postfijo (variable++)*: Devuelve el valor de la variable y luego realiza el incremento.
 - ▶ *Incremento Prefijo (++variable)*: Realiza el incremento y luego devuelve el valor de la variable.

Operadores (Cont.)

- ▶ **Operadores Aritméticos:** Continuando con lo anterior:
 - ▶ *Decremento (--):* Disminuye el valor de una variable en uno.
 - ▶ *Decremento Postfijo (variable--):* Devuelve el valor de la variable y luego realiza el decremento.
 - ▶ *Decremento Prefijo (--variable):* Realiza el decremento y luego devuelve el valor de la variable.
- ▶ **Operadores de Asignación:**
 - ▶ *Asignación (=):* Asigna un valor a una variable.
 - ▶ *Asignación de suma (+=), asignación de resta (-=), y así sucesivamente para *=, /=, y %=:* Realizan la operación y asignan el resultado en un solo paso.

Operadores (Cont.)

▶ Operadores de Comparación:

- ▶ *Igualdad (==)*: Verifica si dos valores son iguales, con conversión de tipo.
- ▶ *Igualdad estricta (===)*: Verifica si dos valores son iguales, sin conversión de tipo.
- ▶ *Desigualdad (!=)*: Verifica si dos valores no son iguales, con conversión de tipo.
- ▶ *Desigualdad estricta (!==)*: Verifica si dos valores no son iguales, sin conversión de tipo.
- ▶ *Mayor que (>), menor que (<), mayor o igual que (>=), menor o igual que (<=)*.

Operadores (Cont.)

▶ Operadores Lógicos:

- ▶ *AND* (&&): Devuelve true si ambos operandos son verdaderos.
- ▶ *OR* (||): Devuelve true si alguno de los operandos es verdadero.
- ▶ *NOT* (!): Niega el valor de verdad de su operando.

▶ Operadores de Cadena:

- ▶ *Concatenación* (+): Une dos cadenas de texto en una.

▶ Operadores de Tipo:

- ▶ *typeof*: Devuelve una cadena indicando el tipo de una variable.
- ▶ *instanceof*: Verifica si un objeto es instancia de un constructor específico.

Operadores (Cont.)

- ▶ **Operador de Coalescencia de Nulos (??):** Este operador permite asignar un valor predeterminado solo cuando el operando de la izquierda es null o undefined.
Antes de su aparición, era común usar el operador || para lograr un comportamiento similar. Sin embargo, || retorna el operando derecho si el izquierdo es falsy, lo que incluye: 0, "" (cadena vacía), false, NaN (Not a Number), null y undefined.
Esto podía producir resultados no deseados si valores como 0 o "" eran considerados válidos.

```
let nombre = usuario.nombre ?? "Invitado";
```

Operadores (Cont.)

► Operadores Unarios:

- *delete*: Elimina una propiedad de un objeto.
- *void*: Evalúa una expresión y devuelve undefined. Anteriormente, era común ver el operador void en atributos href de elementos <a> en HTML, para ejecutar código JavaScript sin alterar la página actual. Esto se debe a que void garantiza que el enlace no cause una redirección o recarga, dado que el resultado de la expresión es undefined.

```
<a href="javascript:void(alert('Hola mundo'));">Haz clic</a>
```

Operadores (Cont.)

- ▶ **Operador Ternario (? :):** Evalúa una condición y devuelve un valor si es true y otro si es false.

```
condición ? expresion1 : expresion2;
```

- ▶ *condición*: Una expresión que se evalúa. Puede ser cualquier expresión que devuelva true o false.
- ▶ *expresion1*: La expresión que se evalúa y se ejecuta si la condición es true.
- ▶ *expresion2*: La expresión que se evalúa y se ejecuta si la condición es false.

Operadores (Cont.)

- **Operador de Propagación (Spread Operator):** Permite expandir un iterable (como un array) en lugares donde se esperan cero o más argumentos (para llamadas de función) o elementos (para arrays).

```
let partes = ["hombros", "rodillas"];  
let cuerpo = ["cabeza", ...partes, "dedos"];
```

Operadores (Cont.)

- **Operador Resto (Rest Operator):** Combina los argumentos restantes en una función en un array.

```
function suma(...numeros) {  
    return numeros.reduce((total, numero) => total + numero, 0);  
}
```

```
console.log(suma(1, 2)); // 3
```

```
console.log(suma(1, 2, 3, 4, 5)); // 15
```

Operadores (Cont.)

- ▶ **Operador de Desestructuración:** Permite extraer valores de un array o propiedades de un objeto y asignarlos directamente a variables individuales, de manera concisa y legible.

```
let [a, b] = [1, 2];  
let {x, y} = {x: 10, y: 20};  
console.log(a, b, x, y); // 1 2 10 20
```

- ▶ **Operador de Encadenamiento Opcional (?.):** Permite acceder a propiedades anidadas de un objeto sin necesidad de verificar manualmente que cada nivel de la cadena exista. Si alguna parte de la cadena es null o undefined, la expresión devuelve undefined en lugar de lanzar un error.

```
const nombre = persona?.nombre; // "undefined" si  
"persona" es "null" o "undefined"
```

Objetos y Herencia

En JavaScript, un **objeto es una colección de propiedades, donde cada propiedad se define como un par clave-valor**. Las claves son cadenas (o Symbols) y los valores pueden ser de cualquier tipo, incluyendo funciones. Cuando una propiedad tiene como valor una función, se la llama método.

Los objetos en JavaScript son dinámicos, es decir, se pueden crear, modificar y eliminar propiedades en tiempo de ejecución.

Objetos y Herencia (Cont.)

Notación Literal

```
let persona = {  
  nombre: "Ámbar",  
  edad: 10,  
  saludar: function() {  
    console.log("Hola, mi nombre es " + this.nombre);  
  }  
};
```


Objetos y Herencia (Cont.)

Constructor Object

```
let persona = new Object();  
persona.nombre = "Ámbar";  
persona.edad = 10;  
persona.saludar = function() {  
  console.log("Hola, mi nombre es " + this.nombre);  
};
```

Objetos y Herencia (Cont.)

Prototipos

JavaScript es, en esencia, un lenguaje basado en **prototipos**. Esto significa que los objetos no heredan de clases como en otros lenguajes, sino que **heredan directamente de otros objetos**.

Cada objeto tiene una referencia interna llamada `[[Prototype]]`, que apunta a otro objeto.

Cuando intentamos acceder a una propiedad:

- ▶ primero se busca en el propio objeto
- ▶ si no está, se busca en su prototipo
- ▶ y así sucesivamente

Objetos y Herencia (Cont.)

Clases

A partir de ECMAScript 2015 (ES6), JavaScript incorporó la sintaxis de class.

Esto permite escribir código de una forma más familiar para quienes vienen de lenguajes como Java o C#.

Sin embargo, es importante entender algo: **esto no cambia cómo funciona JavaScript internamente.**

Las clases en JavaScript:

- ▶ no reemplazan el modelo de prototipos
- ▶ son una forma más clara de trabajar sobre él

Objetos y Herencia (Cont.)

Clases

```
class Persona {  
  constructor(nombre) {  
    this.nombre = nombre;  
  }  
  saludar() {  
    console.log(`Hola, mi nombre es ${this.nombre}`);  
  }  
}
```

Objetos y Herencia (Cont.)

Clases

```
class Empleado extends Persona {  
  constructor(nombre, puesto) {  
    super(nombre);  
    this.puesto = puesto;  
  }  
  saludar() {  
    super.saludar();  
    console.log(`Mi puesto es ${this.puesto}`);  
  }  
}
```

Objetos y Herencia (Cont.)

Clases

```
const empleado = new Empleado("Miguel", "Desarrollador");  
empleado.saludar(); // Hola, mi nombre es Miguel  
                    // Mi puesto es Desarrollador
```

Manejo de errores en JavaScript

JavaScript permite controlar errores usando try...catch, evitando que el programa se detenga por completo.

¿Cómo funciona?

- ▶ **try:** Contiene el código que queremos ejecutar.
Si ocurre un error, se interrumpe y pasa al catch.
- ▶ **catch:** Se ejecuta cuando hay un error.
Permite manejarlo o mostrar un mensaje.
- ▶ **throw:** Permite generar errores manualmente.
Se usa cuando queremos indicar que algo no es válido.
- ▶ **finally (opcional):** Se ejecuta siempre, haya error o no.
Suele usarse para limpiar o cerrar procesos.

try intenta → catch maneja → finally siempre ejecuta

Manejo de errores en JavaScript (Cont.)

```
try {  
    // algo que falla  
}  
catch (error) {  
    if (error instanceof TypeError) {  
        console.log("Es un error de tipo");  
    } else if (error instanceof SyntaxError) {  
        console.log("Es un error de sintaxis");  
    } else {  
        console.log("Otro tipo de error: ", error.message);  
    }  
}
```


Rompiendo referencias

En JavaScript, cuando se asigna un objeto (incluidos arrays y funciones) de una variable a otra, **ambas variables apuntan a la misma referencia en memoria**. Es decir, no se crea una copia, sino que ambas comparten el mismo objeto.

```
let original = { a: 1 };  
let copia = original;  
copia.a = 2;  
  
console.log(original.a); // 2
```

Rompiendo referencias (Cont.)

¿Cómo evitamos este comportamiento?

- **Copia superficial de arrays:** Se usa el método `.slice()` o el operador de propagación (`...`) para crear una copia independiente del array:

```
let original = [1, 2, 3];  
let copia1 = original.slice();  
let copia2 = [...original];  
copia1.push(4);  
console.log(original); // [1, 2, 3]  
console.log(copia1);   // [1, 2, 3, 4]
```

Rompiendo referencias (Cont.)

¿Cómo evitamos este comportamiento?

- **Copia superficial de objetos:** Para objetos, se usa *Object.assign()* o el operador de propagación (...):

```
let original = { a: 1, b: 2 };  
let copia1 = Object.assign({}, original);  
// o bien:  
let copia2 = { ...original };  
copia1.a = 3;  
console.log(original); // { a: 1, b: 2 }  
console.log(copia1);   // { a: 3, b: 2 }
```

Estas técnicas no copian objetos anidados. Los subobjetos siguen siendo referencias compartidas.

Rompiendo referencias (Cont.)

¿Cómo evitamos este comportamiento?

- **Copia profunda:** Para romper completamente las referencias, incluyendo objetos anidados, se usa una copia profunda. Una técnica común es:

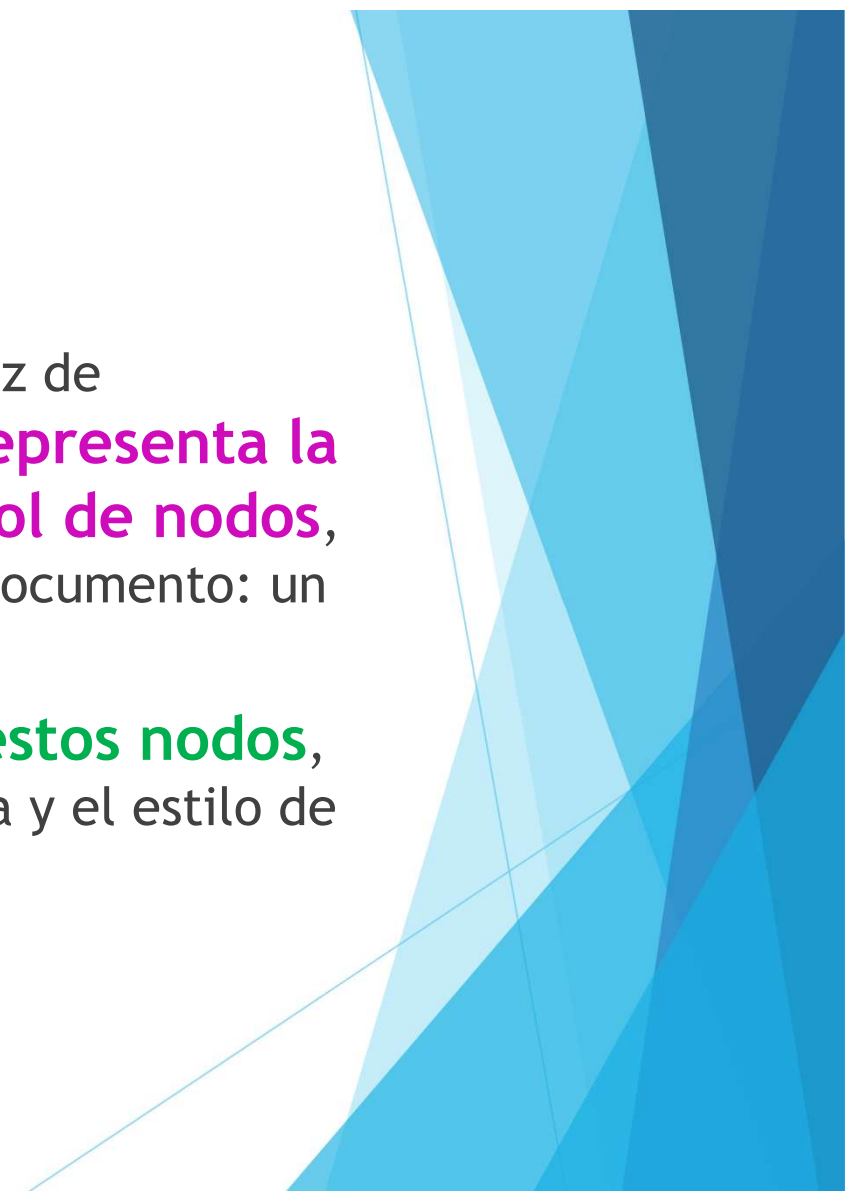
```
let original = { a: 1, b: { c: 2, d: 3 } };
let copiaProfunda = JSON.parse(JSON.stringify(original));
copiaProfunda.b.c = 4;
console.log(original);           // { a: 1, b: { c: 2, d: 3 } }
console.log(copiaProfunda);      // { a: 1, b: { c: 4, d: 3 } }
```

DOM y Eventos

¿Qué es el DOM?

El DOM (Document Object Model) es una interfaz de programación para documentos HTML y XML. **Representa la estructura del documento como un árbol de nodos**, donde cada nodo corresponde a una parte del documento: un elemento, un atributo o un texto.

JavaScript puede acceder y modificar estos nodos, permitiendo cambiar el contenido, la estructura y el estilo de una página Web en tiempo real.



DOM y Eventos (Cont.)

¿Qué son los eventos?

Un evento es una **acción que ocurre en el navegador**. Puede ser generado por:

- ▶ El navegador (por ejemplo, la carga de la página)
- ▶ El usuario (clics, teclas, movimiento del mouse)
- ▶ Otras fuentes (respuestas de red, temporizadores, etc.)

JavaScript puede **escuchar y responder a estos eventos mediante funciones**. Algunos eventos comunes incluyen:

- ▶ **click**: cuando un elemento es clickeado.
- ▶ **submit**: cuando se envía un formulario.
- ▶ **keydown, keyup**: cuando se presiona o suelta una tecla.

DOM y Eventos (Cont.)

¿Qué son los eventos?

Continuando con lo anterior:

- ▶ **input**: cuando se modifica el valor de un campo de formulario.
- ▶ **change**: cuando un campo pierde el foco con un valor distinto.
- ▶ **mouseover, mouseout**: cuando el cursor entra o sale de un elemento.
- ▶ **load**: cuando el documento o un recurso se ha cargado completamente.
- ▶ **scroll**: cuando se hace scroll en un elemento o la ventana.
- ▶ **resize**: cuando se redimensiona la ventana del navegador.

DOM y Eventos (Cont.)

¿Qué son los eventos?

Continuando con lo anterior:

- ▶ **focus:** cuando un elemento (como un `<input>`) recibe el foco.
- ▶ **blur:** cuando el elemento pierde el foco.

DOM y Eventos (Cont.)

Manipulación del DOM

JavaScript permite cambiar el DOM como respuesta a eventos u otras condiciones:

```
// Cambiar el contenido de texto
document.getElementById('mensaje').textContent = '¡Hola
JavaScript!';

// Añadir un nuevo elemento al div
const nuevoElemento = document.createElement('div');
nuevoElemento.textContent = 'Vamos a seguir aprendiendo';
document.getElementById('mensaje').appendChild(nuevoElemento);
```

DOM y Eventos (Cont.)

Añadir Event Listeners

La función **addEventListener** permite ejecutar una función cada vez que se produce el evento especificado.

element.addEventListener(event, function, useCapture);

- ▶ **element:** El nodo del DOM que escucha el evento.
- ▶ **event:** El tipo de evento, por ejemplo, “click”.
- ▶ **function:** La función que se ejecuta cuando ocurre el evento.
- ▶ **useCapture (opcional):** Si es false (valor por defecto), el listener se ejecuta en la fase de burbuja, cuando el evento llega al elemento y sube en el DOM. Si es true, se ejecuta en la fase de captura, antes de alcanzar el elemento objetivo.

DOM y Eventos (Cont.)

Propagación de Eventos

Por defecto, los eventos en JavaScript se manejan en la **fase de burbuja**.

Esto significa que:

- ▶ el evento ocurre en el elemento (por ejemplo, un botón)
- ▶ luego se propaga hacia arriba en el DOM
- ▶ pasando por sus elementos padres (div, body, etc.)

Es como si el evento “subiera” por el árbol.

DOM y Eventos (Cont.)

Propagación de Eventos

También existe una fase anterior, llamada **captura**.

En este caso, el evento:

- ▶ baja desde los elementos más externos
- ▶ hasta llegar al elemento donde ocurre

Se utiliza cuando se necesita:

- ▶ controlar flujo antes del elemento
- ▶ implementar lógica global (como validaciones, bloqueos, etc.)

DOM y Eventos (Cont.)

Propagación de Eventos

Además, existen métodos para detener la propagación:

- ▶ **event.stopPropagation():** detiene la propagación del evento hacia los elementos padres.
- ▶ **event.stopImmediatePropagation():** detiene la propagación y evita que otros listeners en el mismo elemento se ejecuten.

Por ejemplo,

```
<div id="padre">  
  Clic en el botón <button id="hijo">Botón</button>  
</div>
```

DOM y Eventos (Cont.)

Propagación de Eventos

```
<script>
document.getElementById('padre').addEventListener('click', (event) => {
  console.log('Captura en el padre');
  const permitido = confirm('¿Permitir acción?');
  if (!permitido) {
    event.stopPropagation();
    console.log('Evento detenido en captura');
  }
}, true);
```

DOM y Eventos (Cont.)

Propagación de Eventos

```
document.getElementById('hijo').addEventListener('click', (event) => {  
    console.log('Click en el botón hijo');  
});  
</script>
```

DOM y Eventos (Cont.)

Prevención del comportamiento predefinido

Algunos eventos tienen un comportamiento por defecto del navegador que puede no ser deseado en ciertas situaciones. Por ejemplo:

- ▶ Un enlace (<a>) intenta navegar a otra URL.
- ▶ Un formulario (<form>) se envía y recarga la página.
- ▶ Un botón tipo submit dentro de un formulario también envía el formulario automáticamente.

En estos casos, se usa **event.preventDefault()** dentro del listener para anular el comportamiento predeterminado y ejecutar nuestra propia lógica.

DOM y Eventos (Cont.)

Prevención del Comportamiento Predefinido

```
<a href="https://www.ubp.edu.ar" id="enlace">Ir a la UBP</a>
<script>
document.getElementById('enlace').addEventListener('click',
function(event) {
    event.preventDefault();
    console.log('El enlace no se ejecuta');
});
</script>
```

Formularios

Un buen formulario es fundamental para ofrecer una experiencia de usuario clara, accesible y eficiente. Además, es clave para que JavaScript pueda interactuar fácilmente con los datos ingresados por el usuario.

Buenas prácticas al construir formularios:

- ▶ Usar **id únicos** para facilitar el acceso desde JavaScript.
- ▶ **Evitar duplicar id**, ya que debe ser único en todo el documento.
- ▶ Utilizar **name descriptivos** que permitan identificar claramente cada campo al momento de procesar los datos.
- ▶ **Agrupar campos relacionados** (como radio o checkbox) utilizando el mismo name.
- ▶ **Aplicar etiquetas <label>** asociadas a los campos para mejorar la accesibilidad.

Selectores

JavaScript ofrece diferentes métodos para acceder a los elementos del DOM. Elegir el adecuado depende del contexto y del tipo de selección que se desea hacer.

- ▶ **getElementById():**

- ▶ Devuelve un único elemento que coincide con el id especificado.
- ▶ Es el método más rápido y directo para obtener un elemento único.
- ▶ El id debe ser único en todo el documento.

Selectores (Cont.)

- ▶ **getElementsByTagName():**
 - ▶ Devuelve todos los elementos que coinciden con un nombre de etiqueta HTML (como 'div', 'input', 'p', etc.).
 - ▶ El resultado es una colección HTML, por lo que se debe iterar sobre ella si se quiere acceder a cada elemento.
- ▶ **getElementsByClassName():**
 - ▶ Devuelve todos los elementos que contienen la clase o clases especificadas.
 - ▶ También devuelve una colección HTML y permite múltiples coincidencias.

Selectores (Cont.)

▶ `getElementsByName()`:

- ▶ Devuelve una colección de todos los elementos que comparten el mismo atributo **name**.
- ▶ Se utiliza frecuentemente en formularios donde varios campos (como radios o checkboxes) comparten el mismo nombre.
- ▶ El resultado es una colección HTML, por lo que es necesario iterar sobre ella si se quiere acceder a cada elemento.

Selectores (Cont.)

▶ `querySelector()`:

- ▶ Devuelve el primer elemento que coincida con el selector CSS proporcionado (por ejemplo, `#id`, `.clase`, `div input`, etc.).
- ▶ Es muy útil para seleccionar un único elemento cuando se desea precisión con selectores más complejos que un id.

▶ `querySelectorAll()`:

- ▶ Devuelve todos los elementos que coincidan con el selector CSS especificado.
- ▶ El resultado es una `NodeList`, sobre la cual se puede iterar con `forEach` o convertir a array.
- ▶ Ideal para aplicar acciones a múltiples elementos con la misma clase, etiqueta, etc.

Validaciones con HTML5 y Constraint Validation API

Validar los datos que los usuarios ingresan en un formulario es una parte fundamental del desarrollo Web. **Las validaciones permiten mejorar la experiencia del usuario, evitar errores y garantizar que los datos enviados sean útiles y seguros.** HTML5 y JavaScript ofrecen herramientas que trabajan en conjunto para facilitar este proceso.

¿Por qué validar formularios?

- ▶ Prevenir el envío de información incompleta o inválida.
- ▶ Guiar al usuario en la correcta completitud de los campos.
- ▶ Reducir la carga en el servidor al filtrar errores desde el cliente.
- ▶ Hacer la interfaz más interactiva y clara.

Validaciones con HTML5 y Constraint Validation API (Cont.)

HTML5 permite aplicar restricciones directamente desde el HTML, sin necesidad de escribir JavaScript. Algunos atributos útiles son:

- ▶ **required**: El campo no puede estar vacío.
- ▶ **type="email"**: Verifica que el contenido tenga formato de correo electrónico (solo valida si existe un @).
- ▶ **min, max**: Establecen límites para campos numéricos o fechas.
- ▶ **pattern**: Usa expresiones regulares para definir un formato válido.
- ▶ **maxlength, minlength**: Limita la longitud del texto ingresado.

Validaciones con HTML5 y Constraint Validation API (Cont.)

Otras veces, el formulario puede tener definido el atributo **novalidate**, lo que le indica al navegador que no ejecute la validación automática del HTML5 al enviar el formulario.

Esto puede ser útil cuando se desea controlar completamente la validación desde JavaScript, sin que se disparen los mensajes predeterminados del navegador.

```
<form id="form" novalidate>  
  <!-- campos del formulario -->  
</form>
```

Validaciones con HTML5 y Constraint Validation API (Cont.)

Cuando es necesario aplicar reglas más específicas, JavaScript permite detectar errores y mostrar mensajes personalizados usando **Constraint Validation API**. Algunos de sus métodos:

- ▶ **checkValidity()**: Devuelve true si el campo cumple con todas las restricciones HTML5.
- ▶ **validity**: Objeto que indica qué restricción se está violando (como `valueMissing`, `typeMismatch`, etc.).
- ▶ **setCustomValidity()**: Permite reemplazar el mensaje de error predeterminado.
- ▶ **reportValidity()**: Muestra visualmente el mensaje de validación en pantalla y devuelve true o false según si el campo es válido. Es útil para evitar el uso de alertas.

Validaciones con HTML5 y Constraint Validation API (Cont.)

```
<form id="formulario" novalidate method="post">  
  <label for="edad">Edad:</label>  
  <input type="number" id="edad" name="edad" required max="99">  
  <button type="submit">Enviar</button>  
</form>
```

Validaciones con HTML5 y Constraint Validation API (Cont.)

```
document.getElementById('formulario').addEventListener('submit',  
function (event) {  
    const edad = document.getElementById('edad');  
    if (!edad.checkValidity()) {  
        if (edad.validity.valueMissing) {  
            edad.setCustomValidity("Por favor, ingresá tu edad");  
        }  
        else if (edad.validity.rangeOverflow) {  
            edad.setCustomValidity("La edad máxima permitida es 99 años");  
        }  
    }  
}
```

Validaciones con HTML5 y Constraint Validation API (Cont.)

```
else {  
    edad.setCustomValidity("");  
}  
if (!edad.reportValidity()) {  
    event.preventDefault();  
}  
});
```

Web Storage

El almacenamiento Web (Web Storage) proporciona dos mecanismos para **almacenar datos en el navegador**: `localStorage` y `sessionStorage`. Ambos **permiten guardar pares clave-valor** directamente en el navegador del usuario, y forman parte de la API de almacenamiento del lado del cliente.

A diferencia de las cookies, **la información guardada no se envía al servidor en cada solicitud**. Además, ofrecen un espacio de almacenamiento significativamente mayor (alrededor de 5MB frente a los 4KB de las cookies).

Importante: No debe utilizarse para guardar datos sensibles como contraseñas o información personal crítica.

Web Storage: localStorage

- ▶ Los datos persisten indefinidamente, incluso si el usuario cierra el navegador.
- ▶ Ideal para guardar preferencias del usuario, historial, configuración, etc.

```
localStorage.setItem('usuario', 'Ámbar');  
const usuario = localStorage.getItem('usuario');  
console.log(usuario); // Ámbar  
localStorage.removeItem('usuario');  
localStorage.clear(); // Limpiar todo
```

Web Storage: sessionStorage

- ▶ Los datos persisten solo durante la sesión de la pestaña actual.
- ▶ Se eliminan automáticamente al cerrar la pestaña o el navegador.
- ▶ Útil para datos temporales como el estado de navegación en una única sesión.

```
sessionStorage.setItem('estadoPagina', 'activo');  
const estado = sessionStorage.getItem('estadoPagina');  
console.log(estado); // activo  
sessionStorage.removeItem('estadoPagina');  
sessionStorage.clear(); // Limpiar todo
```


Buenas Prácticas

En programación, no alcanza con que el código funcione. También debe ser **claro, reutilizable y fácil de mantener en el tiempo**.

JavaScript permite lograr esto si se siguen algunas buenas prácticas.

- ▶ **Separar la lógica en funciones pequeñas:** Dividir el código en funciones que realicen tareas específicas ayuda a:
 - ▶ mejorar la legibilidad
 - ▶ facilitar pruebas
 - ▶ reutilizar código

Buenas Prácticas (Cont.)

- ▶ **Evitar repetir código (DRY - Don't Repeat Yourself):** Si nos encontramos con el mismo código, probablemente convenga convertir el código en una función. Esto permite:
 - ▶ reducir errores
 - ▶ facilitar cambios futuros
 - ▶ mantener coherencia
- ▶ **Escribir código legible:** El código se lee mucho más de lo que se escribe. Por eso es importante:
 - ▶ usar nombres descriptivos (`calcularTotal` en lugar de `ct`)
 - ▶ mantener un formato consistente
 - ▶ evitar complejidad innecesaria

Buenas Prácticas (Cont.)

- ▶ **Comentarios útiles, no excesivos:** Los comentarios deben explicar el **por qué**, no el cómo.
- ▶ **Manejar errores correctamente:** No asumir que todo va a funcionar siempre. Usar:
 - ▶ validaciones
 - ▶ try...catch cuando sea necesario
- ▶ **No mezclar responsabilidades:** Evitar hacer todo en el mismo lugar. Por ejemplo,
 - ▶ lógica → JavaScript
 - ▶ estructura → HTML
 - ▶ estilos → CSS

Buenas Prácticas (Cont.)

- ▶ **Mantener consistencia:** Usar siempre el mismo estilo:
 - ▶ nombres
 - ▶ indentación
 - ▶ estructura

La consistencia reduce errores y mejora la lectura.



